



Linear Systems & Inversion Algorithms

Module: Phase 1 | Week 3 (Days 5 & 6)



Table of Contents

1. Executive Summary
 2. The Geometric Anchor: The Identity Matrix
 3. Algorithmic Division: Matrix Inversion
 4. Floating-Point Precision and The `-0.0` Anomaly
 5. Application to Machine Learning: The Normal Equation
-

1. Executive Summary

Days 5 and 6 concluded the mathematical foundation phase by addressing the problem of algorithmic division in N-dimensional space. In linear algebra, the concept of "division" does not exist. To solve complex systems of equations—which is the absolute prerequisite for training classical Machine Learning algorithms like Linear Regression—an engineer must utilize Matrix Inversion. This phase successfully implemented the `np.linalg` (Linear Algebra) sub-module to generate Identity constraints, compute algorithmic inverses, and programmatically prove mathematical resolution.

2. The Geometric Anchor: The Identity Matrix

In scalar arithmetic, the number 1 is the multiplicative identity (e.g., $5 \times 1 = 5$). In N-dimensional tensor algebra, this concept is represented by the **Identity Matrix** (I).

2.1 Structural Composition

An Identity Matrix is strictly a square matrix ($N \times N$) where the main diagonal consists entirely of 1s, and all other nodes are 0.

```
identity_matrix = np.eye(3)
```

2.2 Geometrical Implications

When a dataset (Matrix A) is multiplied by the Identity Matrix (I), the geometric coordinates of the data remain perfectly unaltered.

$$A \cdot I = A$$

In Deep Learning architecture, Identity Matrices are heavily utilized in **ResNets (Residual Networks)**. They allow neural network gradients to bypass certain computational layers, effectively multiplying by I to prevent the "Vanishing Gradient Problem" during backpropagation.

3. Algorithmic Division: Matrix Inversion

To solve the algebraic equation $Ax = B$ for x , one cannot compute $x = B/A$. The systemic solution requires multiplying both sides by the **Inverse of A** (A^{-1}).

3.1 The Inversion Constraint

Not all matrices can be inverted. A matrix must be:

1. **Square:** The number of rows must exactly equal the number of columns.
2. **Non-Singular:** The matrix's determinant cannot equal zero. If a matrix has linearly dependent rows (e.g., Row 2 is just Row 1 multiplied by 2), the matrix collapses geometrically and cannot be inverted. Attempting to do so yields a `LinAlgError: Singular matrix`.

3.2 Programmatic Execution

```
inverse_A = np.linalg.inv(matrix_A)
```

NumPy calculates the inverse utilizing highly optimized LAPACK (Linear Algebra PACKage) routines. When A is multiplied by A^{-1} , the result is the Identity Matrix (I). This mathematical proof ensures the inversion was successful.

4. Floating-Point Precision and The `-0.0` Anomaly

During the programmatic proof (`matrix_A @ inverse_A`), the Python output yielded a fractional anomaly: `[-0. 1.]`.

4.1 IEEE 754 Floating-Point Arithmetic

CPUs do not calculate fractions perfectly; they estimate them using binary fractions (IEEE 754 standard). When calculating the inverse of a matrix, the CPU performs millions of microscopic decimal multiplications.

Because of hardware rounding limits, a value that should mathematically be `0.0` is computed as `-0.00000000000000011`. When wrapped in `np.round()`, the UI truncates the decimal but retains the negative sign bit, printing `-0.`

Engineering Protocol: In production ML pipelines, engineers use `np.allclose(A, B)` instead of `A == B` to safely compare matrices, mitigating these microscopic floating-point artifacts.

5. Application to Machine Learning: The Normal Equation

Why is matrix inversion critical for Data Scientists?

It is the exact mathematical engine behind the **Normal Equation** used in Ordinary Least Squares (OLS) Linear Regression.

To find the perfect weights (θ) to predict a dataset without using iterative Gradient Descent, the CPU solves:

$$\theta = (X^T X)^{-1} X^T y$$

By executing `np.linalg.inv()`, the data engineer is literally computing the exact solution to a predictive Machine Learning model from scratch.